

Unit-1

- **Linked-List:** A linked list or a one-way list is a linear collection of data elements called nodes, Where the linear order is given by means of pointers. Each node in the list is divided in two parts: The first part contains the information and the second contains the address of the next node in the list called link field.
- **Operations_of_LL:** 1) Traversing 2) Insertion 3) Deletion 4) Searching
- **Doubly/Two Way linked list** is a list that has two addresses, one to the next node and another to previous node.
- **An array** is a linear data structure. Which is a finite collection of similar data items stored in successive or consecutive memory locations.
- **Types of Arrays**
 - **One dimensional array** stores the data elements in a single row or column.
 - **Two dimensional** array stores the data elements placed in rows and columns.
 - **Multi dimensional** array are an extension of 2d array and use subscripts for indexing.
- **Array operations:** Insertion, Deletion, Traversal, Reversing, Sorting, Searching

Unit-2

- **Stack:** A Stack is a list of elements in which an element may be inserted or deleted only at one end called the top of the stack Lists (LIFO (Last In, First Out)) -1->Stack is full 0->Only one element in Stack N-1->Stack is full N-> Stack is overflow
Ex: Call log in mobile, stack of plate in restaurant
- **Basic operations of stack**
 - “Push” is the term used to insert an element into a stack.
 - “Pop” is the term used to delete an element from stack. etc.
- **Application of Stacks:**
 - Evaluating postfix expressions, Quick Sort Converting Infix to postfix expression, Tower of Hanoi problem, Factorial of a number, Fibonacci series
- **Quick Sort:** Quick sort is an algorithm of the divide and conquer type.that is the problem of sorting a set is reduced to the problem of sorting two smaller sets using two Stacks Lower and Upper.
- **Bubble Sort** is used to rearrange a set of elements in an ascending or descending order.
- **Merge Sort:** It is similar to quick sort that works on the principle of Divide and Conquer.

- **Insertion Sort:** It is sorting technique which sorts the array by shifting elements one by one.
- **Selection Sort:** It selects the smallest elements from an unsorted list in each iteration and places that element at the beginning of the unsorted list.
- **Queue:** A Queue is a linear data structure which follows a particular order FIFO (First In First Out) in which the operations are performed.

Ex: A Ticket Line, CPU Scheduling, Disk Scheduling

UNIT-3

-> **A tree** is a kind of non-linear data structure that is used to represent the data in hierarchical form.

-> **A search tree** is a tree data structure used for locating specific keys from within a set

-> **Binary search trees:** In a Binary search tree, the value of left node must be smaller than the parent node, and the value of right node must be greater than the parent node. This rule is applied recursively to the left and right subtrees of the root.

Advantages of Binary search tree

-> Searching an element in the Binary search tree is easy as we always have a hint that which subtree has the desired element.

-> As compared to array and linked lists, insertion and deletion operations are faster in BST.

-> **AVL trees:** AVL Tree can be defined as height balanced binary search tree in which each node is associated with a balance factor which is calculated by subtracting the height of its right sub-tree from that of its left sub-tree.

-> Tree is said to be balanced if balance factor of each node is in between -1 to 1, otherwise, the tree will be unbalanced and need to be balanced.

-> If balance factor of any node is 1, it means that the left sub-tree is one level higher than the right sub-tree.

-> If balance factor of any node is 0, it means that the left sub-tree and right sub-tree contain equal height.

-> If balance factor of any node is -1, it means that the left sub-tree is one level lower than the right sub-tree.

-> AVL trees are **mostly used for in-memory sorts of sets and dictionaries.**

-> **WHY??** AVL tree controls the height of the binary search tree by not letting it to be skewed.

-> **B-tree:** B-tree is a special type of self-balancing search tree in which each node can contain more than one key and can have more than two children. It is a generalized form of the [binary search tree](#). It is also known as a height-balanced m-way tree.

-> We need for B-tree because it needs lesser time in accessing the physical storage media like a hard disk.

->B tree is used to index the data.

->**Red-Black** tree is a self-balancing binary search tree in which each node contains an extra bit for denoting the colour of the node, either red or black.

- **Property #1:** Red - Black Tree must be a Binary Search Tree.
- **Property #2:** The ROOT node must be coloured BLACK.
- **Property #3:** The children of Red coloured node must be coloured BLACK. (There should not be two consecutive RED nodes).
- **Property #4:** In all the paths of the tree, there should be same number of BLACK coloured nodes.
- **Property #5:** Every new node must be inserted with RED colour.
- **Property #6:** Every leaf (i.e. NULL node) must be coloured BLACK.

->**splay trees:** Splay trees are the self-adjusted binary search trees. It has one extra property that is splaying which makes it unique.

Splaying an element, is the process of bringing it to the root position by performing suitable rotation operations.

->**There are six types of rotations used for splaying:**

1. Zig rotation (Right rotation)
2. Zag rotation (Left rotation)
3. Zig zag (Zig followed by zag)
4. Zag zig (Zag followed by zig)
5. Zig zig (two right rotations)
6. Zag zag (two left rotations)

->**2-3 tree:** A **2-3 Tree** is a tree data structure where every node with children has either two children and one data element or three children and two data elements.

Properties of 2-3 tree:

- Nodes with two children are called 2-nodes. The 2-nodes have one data value and two children
- Nodes with three children are called 3-nodes. The 3-nodes have two data values and three children.
- Data is stored in sorted order.
- It is a balanced tree.
- All the leaf nodes are at same level.
- Each node can either be leaf, 2 node, or 3 node.
- Always insertion is done at leaf.

UNIT-4

->**Heaps:** A heap is a tree-based data structure in which (tree is a complete binary tree) all the nodes of the tree are in a specific order.

-> Heaps are used when the highest or lowest order/priority element needs to be removed.

Max-Heap: In a Max-Heap the key present at the root node must be greatest among the keys present at all of it's children. The same property must be recursively true for all sub-trees in that Binary Tree.

Min-Heap: In a Min-Heap the key present at the root node must be minimum among the keys present at all of its children. The same property must be recursively true for all sub-trees in that Binary Tree

Heap Data Structure Applications

- >Heap is used while implementing a priority queue.
- >Dijkstra's Algorithm
- >Heap Sort

Heap Data Structure Operations

-> **Heapify** is the process of creating a heap data structure from a binary tree. It is used to create a Min-Heap or a Max-Heap.

-> **Insert Element into Heap**

-> **Delete Element from Heap**

-> **Peek operation(Find max/min)** returns the maximum element from Max Heap or minimum element from Min Heap without deleting the node.

-> **Extract-Max/Min:** Extract-Max returns the node with maximum value after removing it from a Max Heap whereas Extract-Min returns the node with minimum after removing it from Min Heap.

-> **priority queue implementation:** A priority queue is a special type of queue in which each element is associated with a priority value.

->Elements are served on the basis of their priority.

->Higher priority elements are served first.

->If elements with the same priority occur, they are served according to their order in the queue.

->**Priority queue can be implemented** using an array, a linked list, a heap data structure, or a binary search tree. Among these data structures, heap data structure provides an efficient implementation of priority queues.

Priority Queue Applications

Some of the applications of a priority queue are:

- Dijkstra's algorithm
- for implementing stack
- for load balancing and interrupt handling in an operating system
- for data compression in Huffman code

->**heap sort:** Heap sort is a comparison-based sorting technique based on Binary Heap data structure. It is similar to selection sort where we first find the minimum element and place the minimum element at the beginning. We repeat the same process for the remaining elements.

->**binomial heaps:** A binomial heap can be defined as the collection of binomial trees that satisfies the heap properties, i.e., min-heap. The min-heap is a heap in which each node has a value lesser than the value of its child nodes. Mainly, Binomial heap is used to implement a priority queue.

A Binomial Tree is a recursively defined tree. A binomial tree of order 0 has 1 node. A Binomial Tree of order k can be constructed by taking two binomial trees of order $k-1$ and making one as leftmost child or other.

->**Complete Binary tree** is a tree in which all the levels are completely filled except the last and all nodes are pushed as far left as possible.

-> **Full Binary Tree** is a tree in which all the nodes have 0 or 2 children.

-> **Fibonacci heaps:** Fibonacci Heap is a collection of trees with min-heap or max-heap property. In a fibonacci heap, a node can have more than two children or no children at all. Also, it has more efficient heap operations than that supported by the binomial and binary heaps.

-> The fibonacci heap is called a **fibonacci** heap because the trees are constructed in a way such that a tree of order n has at least F_{n+2} nodes in it, where F_{n+2} is the $(n + 2)^{\text{th}}$ Fibonacci number.

@ **What_is_direct_addressing?** -> Distinct array position for every possible key

Unit 6 (Hashing)

-> Hashing is a technique to storing and retrieving data in $O(1)$ time. -> A technique for direct search is hashing because address is known

-> It is a searching technique which is independent of number n ($n \rightarrow$ number of record) because here we are not sequentially search the record.

-> It is the process of converting a given key into another value. A **hash function** is used to generate the new value according to a mathematical algorithm. The result of a hash function is known a **hash**.

-> **uniform hashing:** any given element is equally likely to hash into any of the slots available in the array.

-> **Application:** Blockchain, Cryptography (Hashing enables people to get data authorization without knowing the content on the data.)

-> **Hash table** is a structure that maps keys to values.

-> **The hash function calculation has two main criteria:**

- The function H should be quick and easy to compute.
- It should uniformly distribute the hash addresses throughout the set L so that there are minimum number of collisions

Hash Function Problems:

- Keys may not be numeric.
- Number of possible keys is much larger than the space available in table.
- Different keys may map into same location
 - Hash function is not one-to-one $\Rightarrow$ collision.
 - If there are too many collisions, the performance of the hash table will suffer dramatically.

-> **Division Method:** The division method involves mapping a key k into one of m slots by taking the remainder of k divided by m as expressed in the hash function. $H(k) = k \pmod{m}$ or $H(k) = k \pmod{m} + 1$ Where $k \pmod{m}$ denotes the remainder when k is divided by m . The second formula is used when we want hash addresses to range from 1 to m rather than from 0 to $m-1$.

Good values for m are primes not too close to exact powers of 2.

-> **Mid square Method:** In this technique, a seed value is taken and it is squared. Then, some digits from the middle are extracted. These extracted digits form a number which is taken as the new seed. The key k is squared. Then the hash function H is defined by $H(k) = p$ Where p is obtained by deleting digits from both ends of k^2

-> **Folding Method:** It breaks up a key value into precise segments that are added to form a hash value.

The key k is partitioned into number of parts $k_1, k_2, \dots, k_r$ where each part, except the last, has the same number of digits as the required address. Then the parts are added together, ignoring the last carry. That is $H(k) = k_1 + k_2 + \dots + k_r$ Where the leading-digit carries, if any, are ignored. Sometimes, the even numbered parts are each reversed before the addition.

-> **Collision:** When two or more keys can generate the same hash value(not take decision which to store in hash table). **OR** when an element is inserted, it hashes to the same value as an already inserted element, then we have a collision.

->Collision Resolution:

- Separate chaining (open hashing)
- Open addressing (closed hashing)

->**Separate chaining:** This method is used to make each cell of hash table point to a linked list of records that have same hash function value.

Advantages:

- Better space utilization for large items.
- Simple collision handling: searching linked list.
- Overflow: we can store more items than the hash table size.
- Deletion is quick and easy: deletion from the linked list.

Disadvantages:

- Cache performance of chaining is not good as keys are stored using a linked list. ...
- Wastage of Space (Some Parts of hash table are never used)
- If the chain becomes long, then search time can become $O(n)$ in the worst case.
- Uses extra space for links.

-> **Open addressing:** A method for handling collisions. In this, all elements are stored in the hash table itself. So, at any point, the size of the table must be greater than or equal to the total number of keys (Note that we can increase table size by copying old data if needed).

Open Addressing is done in the following ways:

In linear probing,

- When collision occurs, we linearly probe for the next bucket.
- We keep probing until an empty bucket is found.

In quadratic probing,

- When collision occurs, we probe for i^2 th bucket in i^{th} iteration.
- We keep probing until an empty bucket is found.

In double hashing,

- We use another hash function $\text{hash}_2(x)$ and look for $i * \text{hash}_2(x)$ bucket in i^{th} iteration.
- It requires more computation time as two hash functions need to be computed.

In double hashing, there are *two* hash functions. The second hash function is used to provide an *offset* value in case the first function causes a collision.

- The advantage of Double hashing is that it is one of the best form of probing, producing a uniform distribution of records throughout a hash table.
- This technique does not yield any clusters

Unit 5 (Graphs)

->A graph is a non-linear data structure that consists of finite set of vertices (nodes) and a set of edges connecting them.

Application of graph:

Facebook, WWW, Google maps, Flight network, Product Recommendations

->**Connected / Disconnected graph:** A graph is said to be connected if there exists at least one path between every pair of vertices. Otherwise, it is disconnected.

Directed / Undirected graph: In directed graph, each edge has a direction which determines the traversal order. In an undirected graph the edges are unidirectional, with no direction associated with them. Hence, the graph can be traversed in either direction.

->**Weighted graph:** A graph where each edge has a numerical weight assigned to it.

->**Cyclic graph** - A directed graph containing at least one graph cycle. A cyclic graph possessing exactly one (undirected, simple) cycle is called a unicyclic graph.

->**Acyclic graph (DAG)** - A directed graph that is not cyclic.

->**Dense / Sparse graph:** Dense graph is a graph in which the number of edges is close to the maximal number of edges. Sparse graph is a graph in which the number of edges is close to the minimal number of edges.

->**Simple graph** - A graph without loops and without multiple edges between two nodes.

->**Complete graph** - A undirected graph in which every pair of distinct vertices is connected by a unique edge. Every complete graph is also a simple graph.

->**Strongly Connected graph:** A directed graph is called strongly connected if there is a path in each direction between each pair of vertices of the graph.

->A Graph is said to be **labeled** if its edges are assigned a data.

->Following are the most commonly used representations of a graph:

- Adjacency Matrix (**Sequential representation of Graphs**)
- Adjacency List (**Linked Representation of Graph**)
- Incidence Matrix

Adjacency Matrix

- An Adjacency Matrix $A[V][V]$ is a 2D array of size $V \times V$ where V is the number of vertices in a graph.
- In this matrix, both rows and columns represent vertices. This matrix is filled with either 1 (or weight) or 0.
- 1 (or weight) represents that there is an edge from row vertex to column vertex • 0 represents that there is no edge from row vertex to column vertex.

Advantages:

- Easy to understand and implement.
- Adding/Removing an edge takes $O(1)$ time.
- Queries like whether there is an edge from vertex 'u' to vertex 'v' are efficient and can be done $O(1)$.

Disadvantages

- Consumes more space $O(V^2)$. Even if the graph is sparse, it consumes the same space.
- Adding/Removing a vertex (node) is $O(V^2)$ time because there may be many changes in the matrix

Adjacency List: An adjacency list is a collection of unordered lists used to represent a finite graph. Each list describes the set of neighbours of a vertex in the graph.

Advantages:

- Saves space. $O(|V|+|E|)$
- Adding an edge takes $O(1)$ time
- Adding a vertex takes $O(1)$ time

Disadvantages:

- Queries like whether there is an edge from vertex u to vertex v are not efficient and can be done $O(V)$.
- Time taken to remove an edge takes $O(E)$ time.
- Time taken to remove a vertex $O(V+E)$ time.

Incidence matrix:

- An incidence matrix is a matrix where each column represents an edge connected to two vertices.
- In this representation, the graph is represented using a matrix of $V * E$.
- Possible values for undirected graph: 0, 1.
- Possible values for directed graph: 0, 1, -1

Graph Traversal: Technique used for visiting each vertex of the graph.

Why?

- Finding all reachable nodes (for garbage collection)
- Finding the best reachable node (single-player game search) or the minmax best reachable node (two-player game search) .
- Finding the best path through a graph (for routing and map directions)

Classification: Depending on the order in which vertices are visited.

- BFS-Breadth First Search
- DFS-Depth First Search

BFS: BFS is a vertex-based technique for finding the shortest path in the graph. In BFS, one vertex is selected at a time when it is visited and marked then its adjacent are visited and stored in the queue.

Start traversal from a node and explore graph layer wise.

Which Data Structure to use? -> FIFO data structure (Queue)

How to avoid cycles? -> Keep track of visited nodes

Algorithm:

- **Rule 1** – Visit the adjacent unvisited vertex. Mark it as visited. Display it. Insert it in a queue.
- **Rule 2** – If no adjacent vertex is found, remove the first vertex from the queue.
- **Rule 3** – Repeat Rule 1 and Rule 2 until the queue is empty.

Application: Peer to peer networks, Social Networking Websites, GPS Navigation System, Path finding, Broadcasting Network

DFS: The depth-first search (DFS) algorithm starts with the initial node of graph G and goes deeper until we find the goal node or the node with no children.

Which Data Structure to use? -> LIFO data structure (Stack)

How to avoid cycles? -> Keep track of visited nodes

Algorithm:

The step-by-step process to implement the DFS traversal is given as follows -

1. First, create a stack with the total number of vertices in the graph.
2. Now, choose any vertex as the starting point of traversal, and push that vertex into the stack.
3. After that, push a non-visited vertex (adjacent to the vertex on the top of the stack) to the top of the stack.
4. Now, repeat steps 3 and 4 until no vertices are left to visit from the vertex on the stack's top.
5. If no vertex is left, go back and pop a vertex from the stack.
6. Repeat steps 2, 3, and 4 until the stack is empty.

Application: Topological sorting, Scheduling problems, Cycle detection in graphs, solving puzzle with only one solution e.g., maze

->Spanning trees: A spanning tree is a subset of Graph G , which has all the vertices covered with minimum possible number of edges. Hence, a spanning tree does not have cycles and it cannot be disconnected.

What are spanning trees used for?

Applications. Minimum spanning trees have direct applications in the design of networks, including computer networks, telecommunications networks, transportation networks, water supply networks, and electrical grids.

->Network flow problems: In graph theory, a flow network (also known as a transportation network) is a directed graph where each edge has a capacity and each edge receives a flow. The amount of flow on an edge cannot exceed the capacity of the edge.

->Warshall's algorithm for shortest path: Warshall algorithm is used to find all pair shortest path problem from a given weighted graph. As a result of this algorithm, it will generate a matrix, which will represent the minimum distance from any node to all other nodes in the graph.

->Topological sort: Topological sorting for Directed Acyclic Graph (DAG) is a linear ordering of vertices such that for every directed edge $u \rightarrow v$, vertex u comes before v in the ordering. Topological Sorting for a graph is not possible if the graph is not a DAG. Topological Sorting can be done by both DFS as well as BFS.

What is topological sort used for?

Scheduling jobs from given dependencies among Jobs. For example, if some job requires the dependency of some other job, then we can use topological sorting.